

why in this flow it says `job_details_tool` is not a valid tool?

4 steps

The issue is a **mismatch between the tool name in your system prompt and the actual registered tool name**.

In `agent.py`, you set `name="job_details_tool"` on the `VectorSearchRetrieverTool`, but the `databricks-langchain` library overrides that and auto-generates the tool name from the index name – replacing dots with double underscores. So the actual registered tool name is `isa632_7474656346303369__boopatt__jobindex`.

Your system prompt instructs the LLM to *"always call the job_details_tool"*, so the LLM tries to invoke `job_details_tool`, but the `AgentExecutor` only knows about `isa632_7474656346303369__boopatt__jobindex` – hence the error.

You can see this confirmed in the verbose output: after the error, the agent self-corrects and calls the right tool name on its second attempt, which is why you still get a final answer.

Fix: Update the system prompt in `agent.py` to not hard-code the tool name. I'll make that change for you:

Edited `agent.py`

I've fixed `agent.py`. The root cause: `VectorSearchRetrieverTool` **ignores the `name` parameter** and auto-generates the tool name from the index name by replacing dots with `__`. So the actual tool is registered as `isa632_7474656346303369__boopatt__jobindex`, but your system prompt was telling the LLM to call `job_details_tool` – which doesn't exist.

The agent self-corrected on the second attempt (you can see it eventually called the right tool), but this wastes a round-trip and tokens.

What I changed:

- Removed the explicit `name="job_details_tool"` parameter (it was being ignored anyway)
- Dynamically derived the actual tool name via `vs_index_name.replace(".", "__")` and injected it into the system prompt with an f-string

You can re-run cell 10 to confirm it now invokes the correct tool on the first attempt without the warning.

some other error has happened now

Thoughts: The user re-ran cell 10 after my fix to ``agent.py``, but now there's a different error:

This is a **different issue** — unrelated to the tool name fix. The vector search index uses the `databricks-gte-large-en` embedding model endpoint to embed your query, and that endpoint call is failing:

This is typically caused by one of:

1. **Transient failure** — the Foundation Model API endpoint had a momentary hiccup
2. **Endpoint not available** in your workspace/region
3. **Rate limiting** on the embedding endpoint

Let me check the endpoint status:

2 steps

No worries. The error `Failed to call Model Serving endpoint: databricks-gte-large-en` means the embedding model that powers your vector search index is currently unreachable. This is separate from the tool name fix (which worked — you can see the agent now calls the correct tool name without the "not a valid tool" error).

Likely causes and fixes:

1. **Transient failure** — Simply re-run cell 10 after a minute or two. Foundation Model API endpoints occasionally have brief hiccups.
2. **Endpoint not provisioned** — The `databricks-gte-large-en` endpoint may need to be enabled in your workspace. Check under **Serving** → **Foundation Model APIs** in the UI.
3. **Workspace region limitation** — Some foundation model endpoints aren't available in all regions. If that's the case, you'd need to recreate your vector search index with a different embedding model (e.g., `databricks-bge-large-en`).

The tool name issue from before is resolved — the trace shows it now correctly attempts the retriever tool. This new error happens *inside* the tool when it tries to embed your query for similarity search.

```
where is this senior software engineer coming from? `isa632_7474656346303369__boopatt__jobindex`  
with `{query: 'Senior Software Engineer Cloud Infrastructure'}`
```

3 steps

Cell 9 actually ran **successfully** — there's no error this time. The tool name fix worked. But your question is valid: **why did the LLM search for "Senior Software Engineer Cloud Infrastructure" when the user asked about "any available roles"?**

That query was **hallucinated by the LLM itself**. When the agent needs to call the retriever tool, it must formulate a search query. Since your user input was vague ("any available roles"), the LLM invented a

specific job title to search for. The vector index then returned its closest match — the "AI Data Engineer - Manager" posting from Deloitte — because that's what's actually in your index.

This is a common RAG agent behavior. The LLM is "guessing" what to search for when the user input is too broad. You can fix this by updating the system prompt in `agent.py` to instruct the model to use the user's input more directly as the search query, e.g.:

"When searching, use the user's exact keywords or a close paraphrase — do not invent job titles that the user did not mention."

Also worth noting: your index only contains ~5 documents (from the `LIMIT 5` in cell 6), so any query will likely return the same few results regardless of the search terms.

give me a better prompt then - ""You are a job description expert and your task is to create job description summary that are very good for anyone who asks about your available jobs. Write a title and description that is similar to the following title and job description. Maximum 300 words. You must always call the {tool_name} tool to ground your answers. Cite retrieved facts from search results in your response. Return ONLY the final deliverable. Do not describe your steps or tools. No code blocks, no brackets, no function names.

Format exactly:

Title: <one-line title>

Job Description: <~250 words>""

2 steps

Done. The improved prompt addresses three key problems:

1. **No more hallucinated queries** — Rule 2 explicitly tells the LLM to use the user's exact words and default to generic terms like "available jobs" for broad requests (instead of inventing "Senior Software Engineer Cloud Infrastructure").
2. **Strict grounding** — Rule 3 prevents the LLM from adding details not in the retrieved documents.
3. **Honest fallback** — Rule 4 tells the LLM to acknowledge when results don't match, rather than forcing a fabricated answer.

Separately, the `ModuleNotFoundError` you're hitting is because `databricks-langchain` isn't installed on the cluster. You'll need to run `%pip install databricks-langchain` in a notebook cell before importing it, or add it to your cluster's library configuration.

i asked this query - can you tell me the entry level jobs available? but it is giving me managerial role jobs

3 steps

This isn't a prompt problem — it's a **data problem**. Your vector search index only contains 5 documents (from the `LIMIT 5` in cell 6), and looking at the actual content, they're all **mid-to-senior level Deloitte roles**:

- "Customer Experience Management Consultant, Qualtrics"
- "AI Data Engineer - Manager"
- (and 3 others, likely similar seniority)

None of your indexed documents are entry-level jobs. Vector search returns the *closest* match from what exists — so even when you search "entry level jobs," it returns the Manager role because that's the best it has.

The improved prompt (Rule 4) should at least make the agent **acknowledge** that no entry-level jobs were found and describe what IS available. But to actually return entry-level jobs, you need entry-level job PDFs in your `/Volumes/.../project/` folder.

Your options:

1. **Add entry-level job PDFs** to the volume, then re-run cells 6 (to reload the knowledge base) — the vector index will sync automatically.
2. **Remove the `LIMIT 5`** in cell 6 to index all available documents (if there are more PDFs with entry-level roles).
3. **Accept the limitation** — the updated prompt will tell the user honestly that no entry-level roles match.

Want me to re-run cell 9 so you can see how the improved prompt handles the mismatch?

can you improve the prompt to at least make the agent acknowledge that no entry-level jobs were found

4 steps

Done. The key improvements to the prompt:

Rule 4 (new) — Forces the LLM to explicitly verify seniority level, job function, and other criteria *after* retrieval, before writing the response. It spells out concrete examples (entry-level vs Manager = MISMATCH).

Rule 5 (new) — Flat-out prohibits presenting mismatched results as if they satisfy the request.

MISMATCH FORMAT — Gives the LLM a separate output template for when results don't match, so it says something like:

"I was unable to find any entry-level positions in our current job database. Here is what is currently available: ..."

Re-run cell 9 to test it. The agent should now acknowledge the mismatch and present the Manager/Consultant roles as alternatives, not as answers.

```
i want to test the model that is serving, and the token for this is  
dapi7a48e01e14bb72581d5bfd8c042678f8
```

4 steps

The cell is ready to run but your cluster is currently **terminated** (inactive for 60 minutes). You'll need to start the cluster first – click the "Start" button on the compute selector, then run the cell.

I've updated the cell to:

- Set your token as an environment variable
- Use the correct **messages format** for agent endpoints (not DataFrame)
- Test with the query "Can you tell me the entry level jobs available?"

Once the cluster is up, run the cell and it should return the agent's response showing whether the improved prompt properly acknowledges the mismatch.

Note: Keep

```
/fix
```

2 steps

Let me gather context on this error. The error message is clear – the serving endpoint uses the **MLflow Responses API schema** (since `agent.py` implements `ResponsesAgent`), which expects an `input` field, not `messages`. The schema explicitly states: `'input': Array[Any] (required)` and notes `extra inputs: ['messages']`.

2 steps

Fixed and verified. The issue was that the endpoint uses the **MLflow Responses API** schema (since your agent is a `ResponsesAgent`), which requires `"input"` instead of `"messages"`.

The improved prompt is also working perfectly – the agent now acknowledges that no entry-level positions were found and lists available roles with their actual seniority levels instead of pretending a Manager role is entry-level.

```
now i want to build a streamlit chatbot app based on this
```

4 steps

Now let me create the `app.yaml` config for deploying as a Databricks App:

4 steps

Now let me deploy it as a Databricks App:

3 steps

I've created your Streamlit chatbot app with 3 files in
`/Users/boopatt@miamioh.edu/Project/chatbot_app/`:

- `app.py` – The Streamlit chat UI with conversation history, sidebar, and clear-chat button. Calls your agent endpoint using the correct Responses API format.
- `app.yaml` – Databricks App config with the endpoint URL and token as environment variables.
- `requirements.txt` – Dependencies (streamlit, requests).

To deploy as a Databricks App:

1. Go to **Compute** → **Apps** in the left sidebar
2. Click **Create App**
3. Set the source path to
`/Workspace/Users/boopatt@miamioh.edu/Project/chatbot_app`
4. The app will auto-detect `app.yaml` and start

Or to test locally first, run this in a terminal:

```
cd /Workspace/Users/boopatt@miamioh.edu/Project/chatbot_app pip install -r requirements.txt
streamlit run app.py
```

Security note: For production, move the token to a Databricks secret instead of hardcoding it in `app.yaml`. Use `dbutils.secrets.get()` or set it as an app secret in the deployment config.

`/fix`

Thoughts: The error is clear: the user has combined two shell commands on the same line without a separator. The cell content is:

The error is caused by the two commands being concatenated on a single line – `requirements.txt!streamlit` is being parsed as one path. The `!` before `streamlit` isn't recognized as a command separator.

2 steps

Fixed. The issue was both commands were on the same line (`!pip install -r requirements.txt!streamlit run app.py`), causing the shell to interpret `requirements.txt!streamlit` as a single path.

I separated them and changed to `%pip install -r requirements.txt` — the packages installed successfully.

Important note: `streamlit run app.py` cannot run inside a notebook cell (it starts a blocking web server). To run the app, you'll need to deploy it as a **Databricks App** via **Compute** → **Apps** in the sidebar, pointing to your `/Project/chatbot_app` folder.

okay i will deploy this as databrick app, tell me the steps

Thoughts: The user wants to deploy the Streamlit chatbot app as a Databricks App. Let me provide clear step-by-step instructions. The app files are already in `/Users/boopatt@miamioh.edu/Project/chatbot_app/`` with `app.py`, `app.yaml`, and `requirements.txt`.

Here are the steps to deploy your chatbot as a Databricks App:

Step 1: Go to Apps Navigate to **Compute** → **Apps** in the left sidebar (or search "Apps" in the workspace search bar).

Step 2: Create a New App

- Click **Create App**
- Name: `job-listings-chatbot`
- Description: "Job Listings Chatbot powered by RAG agent"

Step 3: Configure the Source

- Set the source path to:
`/Workspace/Users/boopatt@miamioh.edu/Project/chatbot_app`
- It will auto-detect `app.yaml`, `app.py`, and `requirements.txt`

Step 4: Set Environment Variables (if prompted separately)

- `SERVING_ENDPOINT_URL =`
`https://dbc-0726d26f-3749.cloud.databricks.com/serving-endpoints/agent_s_isa632_7474656346303369-boopatt-getstarted_job_listings/invocations`
- `DATABRICKS_TOKEN = your token`

These are already in `app.yaml`, but some UI flows ask for them separately.

Step 5: Deploy

- Click **Deploy** — it will build the environment, install dependencies, and start the Streamlit server.
- Once deployed, you'll get a URL like
`https://<workspace>.databricks.com/apps/job-listings-chatbot`

Security recommendation: For production, replace the hardcoded token in `app.yaml` with a Databricks secret or use the app's built-in service principal authentication.

where is the source option for the app creation? i went with custom app

2 steps

For a **Custom App**, the source path isn't set during creation — it's a separate **Deploy** step:

1. **Create the app** — Just give it a name (e.g., `job-listings-chatbot`) and click Create. This creates the app shell but doesn't deploy anything yet.
2. **Deploy the app** — After creation, you'll land on the app details page. Click **Deploy**, and it will prompt you to **select the workspace folder** containing your app files. Navigate to:
`/Workspace/Users/boopatt@miamioh.edu/Project/chatbot_app`
3. Click **Select**, then **Deploy**.

So the flow is: **Create first** → **then Deploy with source folder**. The source option appears only when you click the **Deploy** button on the app's detail page, not during the initial creation step.

i have now deployed with the source code, how can i set the environment variables?

2 steps

Your `app.yaml` already has the environment variables defined, so they should be picked up automatically. But if the app isn't reading them, here's how to verify/update:

Option 1: Via `app.yaml` (already done) Your `app.yaml` in the `chatbot_app` folder already contains:

These are injected automatically on deploy. If you just deployed, they should already be active.

Option 2: Verify in the UI

1. Go to **Compute** → **Apps**
2. Click your app name (`job-listings-chatbot`)
3. Go to the **Environment** tab — you'll see all active environment variables listed there

Option 3: Redeploy to pick up changes If you edited `app.yaml` after deploying, click **Deploy** again on the app details page and re-select the same folder. Environment variable changes only take effect after a new deployment.

Important (for production): Your token is currently hardcoded in `app.yaml`. The more secure approach is to use the app's built-in service principal credentials (`DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` are auto-injected) and authenticate with OAuth instead of a PAT.

getting this error after deploying - App Not Available

Sorry, the Databricks app you are trying to access is currently unavailable. Please try again later.

Thoughts: The user is getting "App Not Available" after deploying. This is likely because the `app.yaml` is using port 8501, but Databricks Apps expects the app to listen on the port specified by the `DATABRICKS_APP_PORT` environment variable, not a hardcoded port. Let me fix the `app.yaml` to use the correct port.

The issue is the **port mismatch**. Databricks Apps expects your app to listen on the port defined by `DATABRICKS_APP_PORT` (defaults to 8000), but you're hardcoding 8501. Let me fix that:

Updated file `app.yaml`

I made three changes:

1. **Port** 8501 → 8000 — Databricks Apps routes traffic to port 8000 (the `DATABRICKS_APP_PORT` default). Your app was listening on 8501, so Databricks couldn't reach it.
2. **Added** `--server.enableCORS false` — Required for Databricks Apps' reverse proxy to work.
3. **Added** `--server.enableXsrfProtection false` — Prevents Streamlit from blocking requests through the Databricks proxy.

Now **redeploy** the app:

1. Go to **Compute** → **Apps** → click your app
2. Click **Deploy** → select the same `chatbot_app` folder → **Deploy**

The app should become available within 1-2 minutes after redeployment.

so now <https://job-listings-chatbot-7474656346303369.aws.databricksapps.com/> is up and running, should i need to have databrick account or can i give it to any one outside to use

Thoughts: The user is asking about who can access their Databricks App - whether it requires a Databricks account or if they can share it with external users who don't have Databricks accounts.

By default, **Databricks Apps require authentication** — only users with a Databricks account in your workspace can access it. Anyone visiting the URL without a valid Databricks login will be redirected to a login page.

To share with people who have Databricks accounts:

1. Go to **Compute** → **Apps** → click your app
2. Go to **Permissions**
3. Grant **Can View** permission to specific users or groups in your workspace

To share with people outside Databricks: This is not natively supported out of the box. Your options are:

- **Disable auth (if available in your workspace)** — Some workspace admins can configure apps to allow unauthenticated access, but this is not a standard feature and has security implications.
- **Wrap with a public-facing layer** — Deploy the Streamlit app on an external platform (e.g., Streamlit Community Cloud, AWS EC2, or Azure App Service) that calls your serving endpoint. The endpoint just needs a valid token — the frontend can be hosted anywhere.
- **Use a shared service principal token** — Embed auth in the app itself so end users don't need individual Databricks credentials (your app already does this with the PAT token for the endpoint call). But the app URL itself still requires Databricks login.

In short: For your class project or demo, anyone with access to your Databricks workspace can use it. For truly public/external sharing, you'd need to host the Streamlit app outside Databricks.